Deployment Plan

Last updated by | Haroun Ahmady | May 12, 2026 at 2:35 PM PDT

Deployment Plan for Client IQ

Dev → Test → Pre-Prod→ Prod (Software & Data Engineering)

1. Objectives

- Safely deploy changes with automated CI/CD to all environments.
- Support parallel work between software engineering and data engineering
- Minimize risk to production
- Ensure data quality, performance, and security

Recommended Delivery Rhythm

- Standard cadence: 1-week sprint cadence with weekly controlled deployment opportunities.
- Sprint Length: 1 week
- Dev Deployments: Daily / as needed
- Test Deployments: 2–3 times per week or by release candidate
- Pre-Prod Deployments: 2-3 times per week
- Prod Deployments: Weekly or biweekly
- Emergency Hotfixes: Expedited path only

2. Environment Overview

Environment	Purpose	Characteristics
Dev	Development & experimentation	Fast iteration, lower controls
Test (QA/UAT)	Validation and	Prod-like, masked data
Pre-Prod	Controlled validation	Prod-like, stable
Prod	Live system	Highly controlled

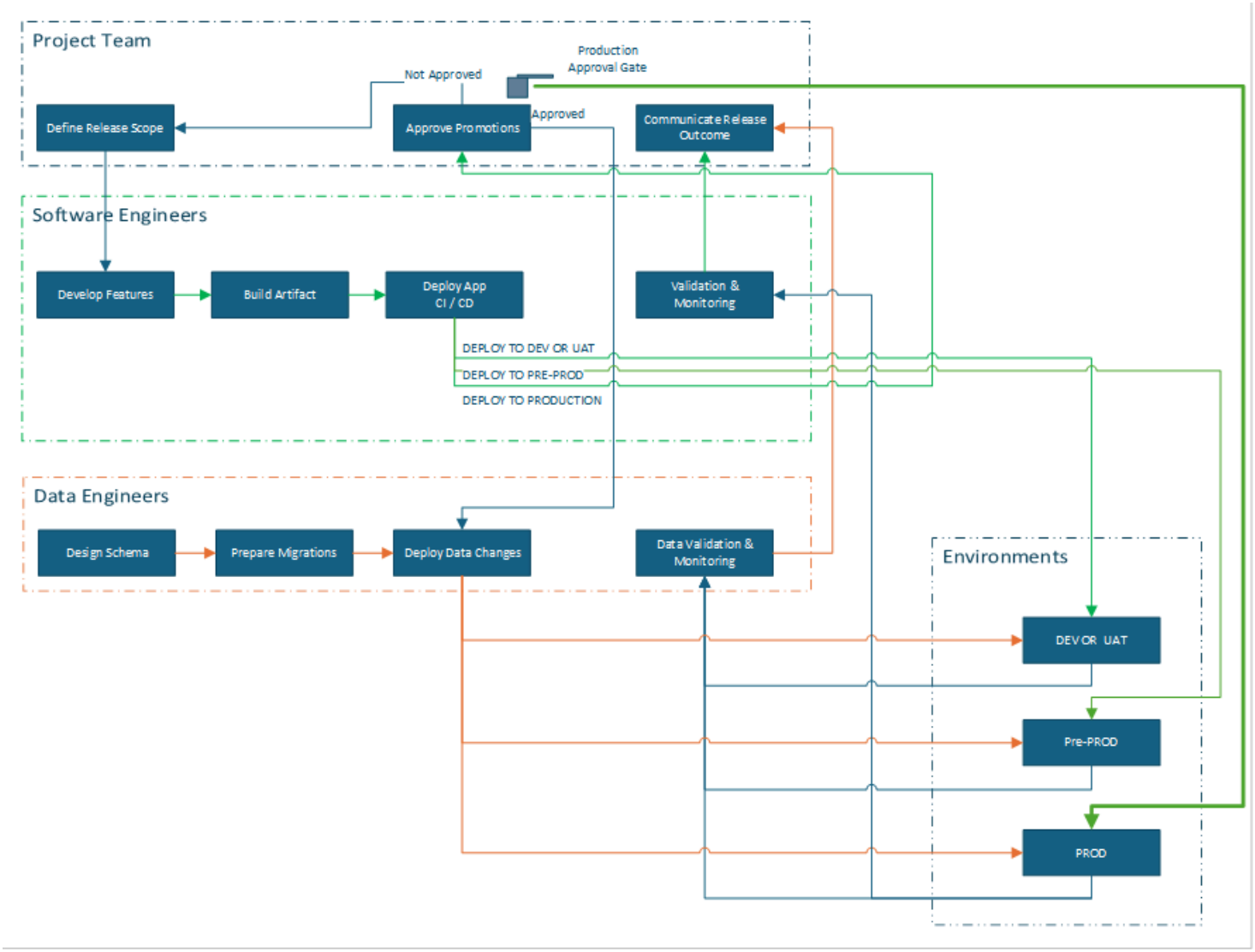
1. **DEV Environment**
   a. Purpose: Active development, unit testing, integration checks, and early validation.
   b. Rules: Developers can deploy frequently; environment may be unstable; masked-production data only.
   c. Used for: feature development, bug fixes, API integration checks, technical validation.
   d. Entry into DEV: ADO work item approved and assigned; acceptance criteria defined; technical design understood.

e. Exit from DEV: Code committed and peer reviewed; unit testing completed; build passed; no critical compile or integration failures; work item updated in ADO.

2. **TEST Environment**
   a. Purpose: Controlled validation environment for QA testing with masked-production data only.
   b. Rules: Only tested build packages are deployed; no active coding directly in Test; environment should remain stable.
   c. Used for: QA functional testing, regression testing, defect validation, integration testing, user validation.
   d. Entry into TEST: Code review complete; CI build passes; deployment package versioned; unit testing evidence exists; deployment notes attached; linked ADO items in proper status.
   e. Exit from TEST: All planned stories completed; critical/high defects resolved or formally waived; test execution complete; release notes finalized; product owner signoff obtained; change approval complete
3. **PRE-PROD Environment**
   a. Purpose: Controlled validation environment for business testing with production data.
   b. Rules: Only approved tested build packages are deployed; no active coding directly in Pre-Prod; environment should remain stable.
   c. Used for: User Acceptance Testing, functional testing, regression testing, defect validation, integration testing, user/business validation.
   d. Entry into TEST: Code review complete; CI build passes; deployment package versioned; unit testing evidence exists; deployment notes attached; linked ADO items in proper status.
   e. Exit from TEST: All planned stories completed; critical/high defects resolved or formally waived; test execution complete; release notes finalized; business/product owner signoff obtained; change approval complete if required
4. **PROD Environment**
   a. Purpose: Live business use.
   b. Rules: Only approved releases allowed; strict deployment window; rollback plan required.
   c. Entry into PROD: Test signoff complete; release notes approved; deployment checklist completed; rollback/backout plan approved; CAB/change approval obtained if applicable; monitoring and support coverage confirmed.

Rules

- Separate infrastructure and databases per environment
- Separate credentials/secrets
- Independent deploy and rollback for each environment



3. Roles & Responsibilities

Software Engineers

- Build deployable artifacts
- Deploy application via CI/CD
- Assess SAST / DAST scan results and remediate high / critical findings.
- Execute verification steps
- Assist with rollbacks

Data Engineers

- Prepare and apply schema changes
- Deploy data pipelines
- Validate data quality and freshness
- Coordinate migrations with releases

Release Lead (Rotating)

- Coordinates deployments
- Runs checklists
- Confirms readiness at each gate
- Records deployment outcomes

QA / Product

- Validate Test/UAT
- Provide production approval

---

4. Source Control & Branching

Branch	Purpose
`develop`	Active development
`release/x.y`	Test & Prod candidate
`main`	Production state

Key Rule:
➡️ The **same commit/build** must go from Test to Prod.

---

5. Dev Environment Deployment

Trigger

- Merge to Dev
- Developer-initiated deploy

Software Engineering (Dev)

1. Pull latest
2. Build application locally or on build server
3. Run unit tests and basic integration checks
4. Deploy to Dev
5. Restart services
6. Verify:
    - App launches
    - Core endpoints respond
    - Logs show no critical errors

✅ No formal approval required

Data Engineering (Dev)

1. Review planned schema changes
2. Apply **non-breaking** migrations
3. Deploy pipelines with Dev configs
4. Run pipelines with sample/synthetic data
5. Validate:
    - Schema integrity
    - Pipeline success
    - Expected row counts

⚠️ No production data in Dev

---

6. Test & Pre-Prod (QA / UAT) Deployment

Trigger

- Dev environment stable
- Release branch (`release/x.y`) created

Pre-Deployment Checklist

- ✅ Code frozen
- ✅ Dev validation complete
- ✅ Schema & pipeline changes reviewed

Software Engineering (Test)

1. Build artifacts from release branch
2. Deploy to Test environment
3. Ensure production-like configuration
4. Restart services
5. Run smoke tests:
    - Login
    - Core workflows
    - API endpoints
6. Hand off to QA/UAT

Data Engineering (Test)

1. Apply Test database migrations (backward-compatible)
2. Deploy Test pipelines
3. Run data validation:

    - Row counts
    - Null checks
    - Key relationships
4. Validate performance where applicable

Approval Gate

- **Ready for Dev**: business requirements, acceptance criteria, priority, owner, dependencies.
- **Ready for Test:** test notes complete, deployment notes complete, build version complete.
- **Ready for Pre-Prod:** build version complete, smoke test notes complete, deployment notes complete, build version complete.
- **Ready for Production:** validation evidence & sign offs (includes Bank Security), critical/high defects resolved, release notes complete, sign off status = complete.

---

7. Production Deployment

Trigger

- Formal CAB approval
- Scheduled deployment window

Pre-Production Checklist (Mandatory)

- ✅ Test environment approved
- ✅ Rollback plan confirmed
- ✅ On-call engineers assigned
- ✅ Monitoring systems ready
- ✅ Backups/snapshots completed (if required)
- ✅ Coordinated deployment with App Services & DBAs (Segregation of Duties)

---

8. Production Deployment Steps

Software Engineering (Prod)

1. Confirm current production version
2. Deploy new version.
3. Validate:
    - Health checks
    - Error rates
4. Monitor closely - Issues / Defects are logged in ADO with proper tagging indicating the environment/stage. (#PROD)

Data Engineering (Prod)

1. Apply schema migrations (non-destructive)
2. Deploy production pipelines
3. Execute smoke data checks:
    - Pipeline completion
    - Freshness
    - Key metrics
4. Monitor for anomalies

⚠️ Breaking data changes must be versioned and phased

---

9. Rollback Strategy (Manual)

Application Rollback

- Re-deploy previous known-good artifact (stable version)
- Disable features via configuration if applicable.
- Restart services

Data Rollback

- Prefer additive, reversible changes
- Use versioned tables or views
- Disable pipelines if needed
- Explicit approval required for rollback actions

---

10. Post-Deployment Activities

- ✅ Validate critical user journeys
- ✅ Confirm data accuracy and freshness
- ✅ Record deployment details:
    - Version
    - Time
    - Issues
    - Rollback actions
- ✅ Communicate release summary

---

11. Required Documentation

Referenced and linked in ADO Wiki.

- Deployment checklists
- Release notes

- Migration scripts
- Rollback instructions
- Known issues log
- Lessons learned
